

EX. NO: 9**SINGLE SOURCE SHORTEST PATH****DATE:****QUESTION :****Write a C program to implement Dijkstra's algorithm to find the shortest path in a weighted graph.****AIM:**

To implement Dijkstra's algorithm to find the shortest path in a weighted graph

ALGORITHM:

1. Start.
2. Initialize the graph with $N = 5$ vertices and assign the edge weights.
3. Initialize $dist[]$ to INF and $vis[]$ to 0 for all vertices.
4. Set the distance of the source vertex 0 to 0.
5. Repeat the following steps for $N-1$ times:
 - o Find the unvisited vertex u having the smallest distance.
 - o Mark vertex u as visited.
 - o Check all adjacent unvisited vertices of u .
 - o If the distance through u is smaller than the current distance, update $dist[]$.
6. After all vertices are processed, $dist[]$ contains the shortest distance from source vertex 0 to every vertex.
7. Display each vertex and its shortest distance from the source.
8. Stop.

PROGRAM:

```
#include <stdio.h>

#define MAX 10
#define INF 9999

void printPath(int parent[], int source, int vertex)
{
    if (vertex == source)
    {
        printf("%c", 'A' + source);
        return;
    }

    printPath(parent, source, parent[vertex]);
    printf(" -> %c", 'A' + vertex);
}
```

```

void dijkstra(int adj[MAX][MAX], int distance[MAX][MAX], int n, int source)
{
    int dist[MAX], visited[MAX], parent[MAX];
    int i, j, min, u;

    for (i = 0; i < n; i++)
    {
        dist[i] = INF;
        visited[i] = 0;
        parent[i] = -1;
    }

    dist[source] = 0;

    for (i = 0; i < n - 1; i++)
    {
        min = INF;
        u = -1;

        for (j = 0; j < n; j++)
        {
            if (!visited[j] && dist[j] < min)
            {
                min = dist[j];
                u = j;
            }
        }

        if (u == -1)
            break;

        visited[u] = 1;

        for (j = 0; j < n; j++)
        {
            if (adj[u][j] == 1 && !visited[j])
            {
                if (dist[u] + distance[u][j] < dist[j])
                {
                    dist[j] = dist[u] + distance[u][j];
                    parent[j] = u;
                }
            }
        }
    }

    printf("\nShortest paths from %c:\n", 'A' + source);

    for (i = 0; i < n; i++)
    {
        if (dist[i] == INF)
        {
            printf("To %c = No path\n", 'A' + i);
        }
    }
}

```

```

else
{
    printf("\n%c to %c = %d",
        'A' + source, 'A' + i, dist[i]);

    if (i == source)
    {
        printf(" (Source)");
    }
    else if (parent[i] == source)
    {
        printf(" directly");
    }
    else
    {
        int path[MAX];
        int count = 0;
        int current = i;

        while (parent[current] != -1 &&
            parent[current] != source)
        {
            path[count++] = parent[current];
            current = parent[current];
        }

        printf(" through ");

        for (j = count - 1; j >= 0; j--)
        {
            printf("%c", 'A' + path[j]);

            if (j != 0)
                printf(" -> ");
        }
    }

    printf("\nPath: ");
    printPath(parent, source, i);
    printf("\n");
}
}

int main()
{
    int adj[MAX][MAX];
    int distance[MAX][MAX];
    int n, i, j;
    char source;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

```

```
printf("\nEnter adjacency matrix (0 or 1):\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        scanf("%d", &adj[i][j]);
    }
}

printf("\nEnter distance for each connected edge:\n");

for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        if (adj[i][j] == 1)
        {
            printf("Distance from %c to %c: ",
                'A' + i, 'A' + j);
            scanf("%d", &distance[i][j]);
        }
        else
        {
            distance[i][j] = 0;
        }
    }
}

printf("\nEnter source vertex (A-%c): ", 'A' + n - 1);
scanf(" %c", &source);

int sourceIndex = source - 'A';

dijkstra(adj, distance, n, sourceIndex);

return 0;
}
```

INPUT AND OUTPUT:

```
[ita2536@mritlin harshan]$ vi ex9.c
[ita2536@mritlin harshan]$ cc ex9.c
[ita2536@mritlin harshan]$ ./a.out
Enter number of vertices: 3

Enter adjacency matrix (0 or 1):

1
0
0
1
1
0
1
0
1

Enter distance for each connected edge:
Distance from A to A: 1
Distance from B to A: 3
Distance from B to B: 5
Distance from C to A: 6
Distance from C to C: 5

Enter source vertex (A-C): B

Shortest paths from B:

B to A = 3 directly
Path: B -> A

B to B = 0 (Source)
Path: B

To C = No path
[ita2536@mritlin harshan]$
```

RESULT:

Thus the implementation of Dijkstra's algorithm to find the shortest path in a weighted graph was executed successfully.